RESEARCH-ARTICLE

Missing Data Imputation with Uncertainty-Driven Network

JIANWEI WANG, UNSW Sydney, Sydney, NSW, Australia

YING ZHANG, Zhejiang Gongshang University, Hangzhou, Zhejiang, China

KAI WANG, Antai College of Economics and Management, Shanghai, China

XUEMIN LIN, Antai College of Economics and Management, Shanghai, China

WENJIE ZHANG, UNSW Sydney, Sydney, NSW, Australia

Proceedings of the ACM on Management of Data, Volume 2, Issue 3 (May 2024)
https://doi.org/10.1145/3654920
EISSN: 2836-6573

Missing Data Imputation with Uncertainty-Driven Network

JIANWEI WANG, University of New South Wales, Australia
YING ZHANG, Zhejiang Gongshang University, China
KAI WANG, Antai College of Economics and Management, Shanghai Jiao Tong University, China
XUEMIN LIN, Antai College of Economics and Management, Shanghai Jiao Tong University, China
WENJIE ZHANG, University of New South Wales, Australia

We study the problem of missing data imputation, which is a fundamental task in the area of data quality that aims to impute the missing data to achieve the completeness of datasets. Though the recent distribution-modeling-based techniques (e.g., distribution generation and distribution matching) can achieve state-of-the-art performance in terms of imputation accuracy, we notice that (1) they deploy a sophisticated deep learning model that tends to be overfitting for missing data imputation; (2) they directly rely on a global data distribution while overlooking the local information.

Driven by the inherent variability in both missing data and missing mechanisms, in this paper, we explore the uncertain nature of this task and aim to address the limitations of existing works by proposing an u**N**certainty-driven netw**O**rk for **M**issing data **I**mputation, termed **NOMI**. NOMI has three key components, i.e., the retrieval module, the neural network gaussian process imputator (NNGPI) and the uncertainty-based calibration module. NOMI runs these components sequentially and in an iterative manner to achieve a better imputation performance. Specifically, in the retrieval module, NOMI retrieves local neighbors of the incomplete data samples based on the pre-defined similarity metric. Subsequently, we design NNGPI that merges the advantages of both the Gaussian Process and the universal approximation capacity of neural networks. NNGPI models the uncertainty by learning the posterior distribution over the data to impute missing values while alleviating the overfitting issue. Moreover, we further propose an uncertainty-based calibration module that utilizes the uncertainty of the imputator on its prediction to help the retrieval module obtain more reliable local information, thereby further enhancing the imputation performance. We also demonstrate that our NOMI can be reformulated as an instance of the well-known Expectation Maximization (EM) algorithm, highlighting the strong theoretical foundation of our proposed methods. Extensive experiments are conducted over 12 real-world datasets. The results demonstrate the excellent performance of NOMI in terms of both accuracy and efficiency.

CCS Concepts: • **Information systems → Data cleaning**.

Additional Key Words and Phrases: Missing Data Imputation; Neural Network Gaussian Process

ACM Reference Format:
Jianwei Wang, Ying Zhang, Kai Wang, Xuemin Lin, and Wenjie Zhang. 2024. Missing Data Imputation with Uncertainty-Driven Network. *Proc. ACM Manag. Data* 2, 3 (SIGMOD), Article 117 (June 2024), 25 pages. https://doi.org/10.1145/3654920

Authors' addresses: Jianwei Wang, University of New South Wales, Australia, jianwei.wang1@unsw.edu.au; Ying Zhang, Zhejiang Gongshang University, China, ying.zhang@zjgsu.edu.cn; Kai Wang, Antai College of Economics and Management, Shanghai Jiao Tong University, China, w.kai@sjtu.edu.cn; Xuemin Lin, Antai College of Economics and Management, Shanghai Jiao Tong University, China, xuemin.lin@sjtu.edu.cn; Wenjie Zhang, University of New South Wales, Australia, wenjie.zhang@unsw.edu.au.

ACM 2836-6573/2024/6-ART117
https://doi.org/10.1145/3654920

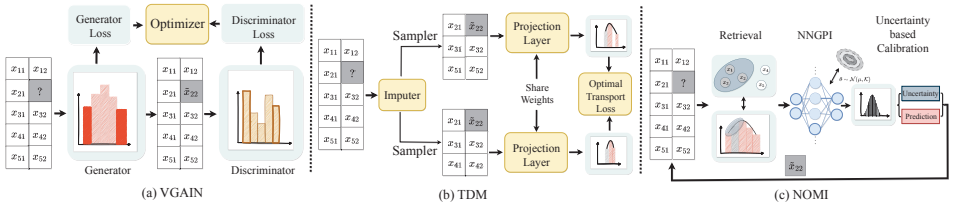


Fig. 1. Frameworks of VGAIN, TDM and NOMI

1 INTRODUCTION

The missing data problem is ubiquitous in many applications due to a variety of reasons such as the poor quality of data collection and privacy issues. It is well known that the completeness of the data is very important in various data analytic tasks. For instance, most deep learning models and algorithms are data-driven which learn to make decisions from data [3]. Hence, data quality has been widely studied to prevent models from falling prey to the garbage in, garbage out (GIGO) syndrome [24]. To properly handle the missing data in the raw dataset, missing data imputation (a.k.a. missing value imputation) has been proposed to predict the missing value using the information in the observed data samples and features to achieve the completeness of the data. Given the importance and popularity of missing data imputation, a spectrum of algorithms [1, 4, 7, 9, 11, 15, 16, 29–31, 33, 36, 42, 45–47, 49, 51, 51, 52, 54, 55] have been proposed. Please refer to the survey papers [25, 34] for more details.

Existing solutions. There is a long history of study for the problem of missing data imputation. In the early stage, the statistics-based imputation (e.g.,[7, 36]) methods and similarity-based methods (e.g.,[1, 16, 47]) are proposed in the literature which are straightforward and intuitive. However, the simple substitution or aggregation lacks the capability to accurately fit the data and thus results in a subpar performance. Some traditional machine learning methods have been adapted for this task to better model the data, leading to compression-based methods (e.g.,[15, 30]) and regression-based methods (e.g.,[9, 11]). Compression-based methods first decompose and then reconstruct the data matrix. Nonetheless, the process of decomposition is time-consuming. Additionally, these methods rely on linearity assumptions, which may fail to capture intricate relationships or patterns in the data. Regression-based imputation methods aim to model relations between observed and missing features. However, the weak correlations among some features present challenges for the regression-based imputation methods.

Recently, a promising research trend is emerging in the field, focusing on the imputation of missing data through distribution modeling techniques. This research direction seeks to learn to capture the underlying data distribution, falling into two primary categories: distribution-generation-based imputation methods (e.g., [9, 11, 29, 31, 45, 49, 54]) and distribution-matching-based imputation methods (e.g., [37, 57]). The distribution-generation-based imputation methods, with VGAIN [34] as a prominent representative depicted in Figure 1(a), apply deep generative models such as auto-encoder (AE) and the generative adversarial net (GAN) to learn the global distribution of the data. Then, the missing data is imputed such that the manipulated data can well fit the learned distribution, resulting in a good performance for missing data imputation. The distribution-matching-based imputation methods, with TDM [57] as a prominent representative depicted in Figure 1(b), captures the idea that any two batches of data in the raw dataset come from the same global distribution and their corresponding distributions should be matched in the original data space [37] or in the latent space [57]. Hence, a neural network is employed to impute the missing data, ensuring that the optimal transport distance between empirical distributions of the imputed data batches is minimized.

Motivations. As demonstrated in the recent survey paper [34] and in [57], VGAIN and TDM show an impressive performance regarding accuracy. However, two primary limitations remain.

Firstly, as highlighted in [26, 41, 59], these distribution-modeling-based approaches deploy a sophisticated deep learning (DL) model, which inherently introduces the risk of overfitting, especially for the task of missing data imputation. When training a DL model for missing data imputation, the learned model may contain a large number of parameters that enable it to capture complex patterns in the training data. However, this can result in the model simply memorizing the training data rather than learning to generalize well to new test data, leading to an inaccurate imputation. Additionally, when the data missing mechanisms are unstructured, the model may overfit specific missing mechanisms present in the training data, thereby struggling to adapt to diverse missing scenarios.

Secondly, these distribution-modeling-based approaches heavily rely on the global distribution to impute the missing data, while the local information remains insufficiently explored. In other words, they impute missing values depending on the underlying global distribution which is obtained from the training samples. However, data samples within one dataset can come from diverse sources, and the global distribution tends to overlook the differences and uniqueness among individual samples locally [58]. Such local information can usually offer valuable context about specific subsets or regions within the global distribution, enabling a more fine-grained and precise imputation.

Challenges. To design an effective imputation method, two main challenges exist below.

Challenge I: How to design an effective and efficient imputator that can alleviate the overfitting problem? The multifaceted nature of the missing data and missing mechanism complicates the overfitting issue. One straightforward way to alleviate the overfitting problem is to use some regularization techniques [19, 60]. However, it is difficult for these regularization techniques to handle complex relations. Another promising direction is to utilize the Bayesian Deep Learning (BDL) models [48, 50]. The BDL methods model uncertainty in parameters and learn a posterior distribution which is effective for alleviating the overfitting problem. However, the traditional BDL needs to specify a prior distribution, which limits the power and flexibility of the model. Moreover, the traditional BDL uses sampling methods (e.g., Markov Chain Monte Carlo [10]) to approximate the posterior distribution, which is a time-consuming process. Thus, how to impute missing data with efficiency and effectiveness while mitigating the overfitting problem is challenging.

Challenge II: How to find reliable local information in the context of missing data? The presence of missing data increases the complexity of local information search. One straightforward way to obtain the local information is directly by the K Nearest Neighbor (KNN) as in similarity-based methods [1, 16, 47] and in [49]. However, the existence of missing data makes the nearest neighbor search inaccurate. Another advanced approach is to run the KNN search after imputation and for multiple iterations [5, 56]. However, the error may propagate and accumulate with each iteration for imputations with large deviations. Therefore, it is challenging to find reliable local information in the context of missing data.

Our solutions. We observe that uncertainty is pervasive in missing data imputation as the intrinsic variability of missing data and missing mechanisms. Driven by the aforementioned challenges and uncertain nature of missing data imputation, we propose an u**N**certainty-driven netw**O**rk for **M**issing data **I**mputation, termed **NOMI**, as illustrated in Figure 1(c). NOMI runs three key components, i.e., retrieval module, neural network gaussian process imputator (NNGPI) and uncertainty-based calibration module, in an iterative manner. These components are interconnected, with the predictions and uncertainty estimated by NNGPI assisting the uncertainty-based calibration and retrieval module in identifying more accurate and reliable local information. In turn, the reliable local information enhances the imputation performance of NNGPI.

(1) Neural Network Gaussian Process Imputator (NNGPI). We devise NNGPI which is based on neural network gaussian process [23] to address *Challenge I*. NNGPI has an infinite number of neurons in its hidden layer and performs equivalent to the Gaussian Process (GP). By functioning as GP, NNGPI quantifies uncertainty and makes probabilistic predictions by considering a range of plausible functions, alleviating overfitting by avoiding overconfidence in specific predictions. Besides retaining the favorable properties of the GP, NNGPI also possesses the universal approximation capabilities of neural networks. By harnessing the representation learning capabilities of neural networks, NNGPI effectively captures complex patterns and structures in data. In contrast to traditional BDL models, NNGPI utilizes neural networks to approximate complex functions, offering flexibility to adapt to diverse data distributions without the need for a specific parametric form for the prior distribution. Furthermore, the presence of an analytical solution in NNGPI enables efficient computations and faster training when compared to traditional BDL models.

(2) Retrieval Module && Uncertainty-based Calibration. To find reliable and accurate local information, we introduce an uncertainty-based calibration technique besides the retrieval module. To be more specific, we leverage the output of NNGPI to calibrate the dataset, which takes into consideration the learned uncertainty, the predicted value in the current iteration and the value predicted in the previous iteration. The calibration prioritizes the prediction in the current iteration when the uncertainty is low, and prioritizes the prediction in the previous iteration when the uncertainty is high. In this way, NOMI can update the dataset with a high level of confidence, thereby mitigating the issue of imputation error accumulation when obtaining local information. In the retrieval module, we retrieve its top-K nearest neighbors based on a pre-defined metric (e.g., L_2 distance in our experiments) to capture the local information. NOMI then sequentially applies retrieval module, NNGPI and uncertainty-based data calibration techniques in multiple iterations, fostering a mutually beneficial relationship among the NNGPI phase and the local information retrieval phase, thereby enhancing the overall performance of imputation. Furthermore, we demonstrate that the NOMI framework can be naturally interpreted as an instance of the Expectation Maximization (EM) algorithm [35]. This correspondence highlights the connection between NOMI and the well-established EM algorithm, enabling NOMI to inherit desirable properties of the EM algorithm, such as the progressively improved imputation accuracy in NNGPI.

Contributions. The key contributions are summarized as follows.

- We propose a new framework, namely NOMI, to significantly enhance the performance of missing data imputation. It combines the power of retrieval module, NNGPI and uncertainty-based data calibration in a series of iterative steps.
- We design NNGPI, an effective and efficient imputation technique based on uncertainty modeling. It merges the powerful approximation capabilities of neural networks with the desirable characteristics of the Gaussian Process.
- We devise an uncertainty-based calibration module and retrieval module to locate more reliable local information. The learned uncertainty is utilized to effectively balance the prediction from different iterations.
- We demonstrate that the NOMI framework can be understood from the perspective of the EM algorithm, thereby highlighting the theoretical foundation and nice properties of NOMI inherited from the EM algorithm.
- Extensive experiments are conducted over 10 public datasets plus 2 million-scale large datasets. The results demonstrate the superior accuracy and efficiency of NOMI. Regarding accuracy, it achieves an average RMSE reduction of approximately 24.58% and 56.64%, when compared to VGAIN and TDM respectively on the first 10 datasets. In terms of efficiency, it achieves impressive

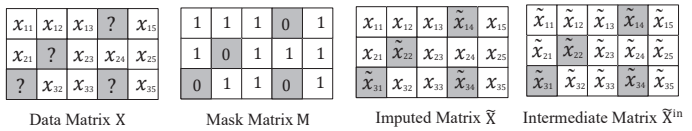


Fig. 2. Comparison of different matrices

Table 1. Symbols and Descriptions

Notation	Description
$X \in \mathbb{R}^{n \times d}$	data matrix with n data samples and d features
$M \in \{0,1\}^{n \times d}$	the mask matrix
$\tilde{X} \in \mathbb{R}^{n \times d}$	the imputed matrix
$\tilde{X}^{in} \in \mathbb{R}^{n \times d}$	the intermediate matrix
$x_i = \{x_{i1}, x_{i2}, \cdots, x_{id}\}$	i-th sample of X
$L_2\left(x_i, x_j\right)$	L_2 distance between x_i and x_j
$\mathcal{G}, \mathcal{D}$	the Generator and Discriminator
$\mathcal{N}(\mu, \sigma)$	the Gaussian distribution

speedups of 8.22× and 46.25× compared to VGAIN and TDM respectively on the first 10 datasets. NOMI also exhibits an outstanding performance over the large datasets.

Roadmap. Section 2 introduces preliminaries. Section 3 presents the framework with detailed theoretical analysis in Section 4. Section 5 reports experimental results. Section 6 reviews related works. Section 7 concludes this paper.

2 PRELIMINARIES

In this section, we first introduce some basic concepts and then formally define the task of missing data imputation. Next, state-of-the-art techniques for missing data imputation are presented.

2.1 Problem Definition

In this paper, the input dataset consists of n data samples where each data sample is a d-dimensional vector (i.e., with d features). It is stored as a data matrix, denoted by $X \in \mathbb{R}^{n \times d}$. For the i-th data sample $x_i = \{x_{i1}, x_{i2}, \cdots, x_{id}\}$ in X, a mask vector $m_i \in \{0,1\}^d$ is used to denote the missing components. The j-th feature of m_i, termed as m_{ij}, equals to 1 if x_{ij} is observed. Otherwise, x_{ij} is missing with $m_{ij} = 0$. When the context is clear, we use x to represent one data sample of the dataset X. All the mask vectors are stacked as a mask matrix $M \in R^{n \times d}$. The frequently used notations are summarized in Table 1.

Problem statement. The task of *missing data imputation* aims to impute the unobserved elements in the data matrix X, i.e., m_{ij} being 0 in the mask matrix M, and make the imputed matrix as close to the real complete dataset (if it exists) as possible. The imputed matrix is denoted by $\tilde{X}$.

Example 1. *Figure 2 gives an example of a data matrix $X \in \mathbb{R}^{3\times5}$, the corresponding mask matrix $M \in \mathbb{R}^{3\times5}$, the imputed matrix $\tilde{X} \in \mathbb{R}^{3\times5}$, and the intermediate matrix $\tilde{X}^{in} \in \mathbb{R}^{3\times5}$ where the all the elements are predicted by the model. The dataset contains three data samples and each data sample is a 5-dimensional vector.*

2.2 State-of-the-arts

As demonstrated in the recent work [57] and the recent survey paper [34] for the problem of missing data imputation, authors demonstrated that VGAIN and TDM achieved the best performance in

terms of accuracy among the distribution-generation-based methods and distribution-match-based methods, respectively.

The framework of VGAIN. VGAIN uses two components, i.e., the generator and the discriminator, to learn the latent distribution of the training samples. Specifically, given the data matrix X, mask matrix M and noise matrix $Z \in \mathbb{R}^{n\times d}$ as inputs, the generator $\mathcal{G}$ outputs the imputed matrix according to the learned distribution:

$$\begin{aligned} \tilde{X}^{in} &= \mathcal{G}(X, M, (1-M)\odot Z) \\ \tilde{X} &= M \odot X + (1-M)\odot \tilde{X}^{in} \end{aligned} \tag{1}$$

where $\odot$ denotes element-wise multiplication.

The discriminator is designed to determine whether the imputed data is generated or from the original distribution. The discriminator is modeled as $\mathcal{D} : \tilde{X}, H \rightarrow [0,1]^{n\times d}$ where $\mathcal{D}(\tilde{X}, H)_{ij}$ is the probability that the j-th component of $\tilde{x}_i$ is observed and H is the hint matrix which contains a fraction of the mask matrix. The model is trained in an adversarial manner, wherein $\mathcal{D}$ is trained to maximize the probability of accurately predicting M, while $\mathcal{G}$ is simultaneously trained to minimize the probability of $\mathcal{D}$ predicting M. The reconstruction loss is also integrated for training.

The framework of TDM. TDM aims to impute any two batches of data X^1 and X^2 such that the empirical distributions of the imputed data are matched in the latent space. To do so, it aims to optimize the following loss:

$$\underset{X^{1\cup 2}[M^{1\cup 2}],\theta}{\arg\min}\ W_2^2\left(\mu\left(f_\theta(X^1)\right), \mu\left(f_\theta(X^2)\right)\right) \tag{2}$$

where $\mu(X^1)$ denotes the empirical measure associated to the samples in X^1. $X^{1\cup 2}$ is the union of X^1 and X^2, i.e., the unique data samples of the two batches. $f_\theta(\cdot)$ is a deep transformation layer that projects the data to the latent space. And $W_2^2(\cdot)$ is the 2-Wasserstein distance to measure the distance between two distributions.

Both approaches utilize complex deep learning models (e.g., the generator and discriminator in VGAIN and the deep transformation layer in TDM which are all characterized by traditional neural networks), consequently posing a potential risk of overfitting. Additionally, these methods heavily rely on the global data distribution learned from the training datasets while the local information remains insufficiently explored. These limitations motivate us to design NOMI to enhance the performance of missing data imputation.

3 METHODOLOGY OF NOMI

In this section, we present the detailed techniques of NOMI which is designed for the problem of missing data imputation. NOMI consists of three key components, including the retrieval module, the NNGPI and the uncertainty-based calibration module. The execution of NOMI involves running these three components sequentially and iteratively to achieve a better imputation performance. The detailed techniques for the three components are in Section 3.1, Section 3.2 and Section 3.3, respectively. The learning objective is presented in Section 3.4. Section 3.5 provides the overall forward propagation procedure of the NOMI framework.

3.1 Retrieval and Input Construction

We exploit the nearest neighbor search [27, 53] method to retrieve the local information. It calculates the pairwise similarity between the input query sample and each data sample in the training dataset given a similarity metric. We utilize the L_2 similarity measure for illustration in this part. We also

provide additional findings on alternative similarity functions such as the cosine similarity and the L_1 similarity in the Exp-12 of Section 5. Given $x_i, x_j \in X$, the L_2 similarity is measured as follows:

$$S\left(x_i, x_j\right) = \frac{1}{L_2\left(x_i, x_j\right)} = \frac{1}{\sqrt{\sum_{p=1}^{d}\left(x_{ip} - x_{jp}\right)^2}} \tag{3}$$

Note that, the L_2 similarity and L_2 distance are inversely proportional. Then, the data samples with the top-K highest similarity score are selected to construct the neighbor set $N(x_i)$. Here, K represents the size of the neighbor set and is configured to 10 based on the experimental findings in Exp-7 of Section 5.

$$\text{idx} = \text{top\_rank}\left(S\left(x_i, X\right), K\right) \tag{4}$$

where $\text{top\_rank}(\cdot, K)$ selects the index of the sample with its corresponding similarity score being the top-K highest w.r.t. the query. Then, the input that is augmented by the local information is constructed as follows:

$$x_i = x_i || \{S^N\left(x_i, x_j\right) \times x_j, \forall j \in \text{idx}\} \tag{5}$$

Where $S^N\left(x_i, x_j\right)$ is the normalized value of $S\left(x_i, x_j\right)$, and $||$ indicates the concatenation operation. The augmented process is conducted for both the training samples and test samples. By incorporating similarity as a weighting factor, we adaptively assign higher weights to closer neighbors, facilitating a more effective and comprehensive capture of local information. We implement the retrieval phase by HNSW [28] which is an efficient algorithm for approximate K nearest neighbor (AKNN) search. Compared with the exact nearest neighbor search algorithms, AKNN by HNSW has much better efficiency and competitive accuracy.

3.2 Neural Network Gaussian Process Imputator

In this part, we first introduce Bayesian learning with its differences compared to traditional deep learning and the Gaussian Process following [59]. Then, we present the details of NNGPI.

In standard deep learning, a model consists of various neural network layers, e.g., fully connected layer [13], convolutional layer [20, 62] and the rectified linear function ReLU [38]. Benefiting from the stack of various layers and the back-propagation [44], the model can be used to approximate the function $f_w(\cdot)$, parameterized by w, between the input x and the target t. Given the training data $x \in X$, the model learns to optimize the learning objective $\mathcal{L}(f_w(x), t)$ between the predicted result $f_w(x)$ and the training target $t \in T$. When the model converges, its parameters w are determined by the following equation:

$$\hat{w} = \arg\min_{w} \mathcal{L}(f_w(x), t) \tag{6}$$

Bayesian deep learning, on the other hand, treats the parameters of the neural network as random variables. The parameter w of the neural network is not deterministic but sampled from a prior distribution $P(w)$ and the distribution is learned by marginalization. Given the training dataset $\mathcal{D} = (X, T)$, Bayesian deep learning learns $p(w|X, T)$ (i.e., the posterior probability of w) given X, T based on Bayes' theorem. The Bayes' theorem is formulated as:

$$P(w|X, T) = \frac{P(X, T|w) * P(w)}{P(X, T)} \tag{7}$$

where $P(X, T|w)$ is the likelihood probability of X, T given the weight w, $P(w)$ is the prior probability of the weight w, and $P(X, T)$ is the marginal probability of training dataset X, T.

In the inference stage, the standard neural network estimates the deterministic target by model inference as follows:

$$P(t_*|x_*, \mathcal{D}) = P(t_*|x_*, \hat{w}) \tag{8}$$

While, Bayesian deep learning estimates the predictive distribution $P(t_*|x_*, \mathcal{D})$ for a given test data x_* by the integral of the posterior probability:

$$\begin{aligned} P(t_*|x_*, \mathcal{D}) &= \int P(t_*|x_*, X, T)\, P(T|\mathcal{D}) dT \\ &= \int P(t_*|x_*, w)\, P(w|\mathcal{D}) dw \end{aligned} \tag{9}$$

Bayesian deep learning makes a prediction by synthesizing the outputs of the neural network parameterized by all the potential weights, with each output being weighted by the posterior of the parameters $P(w|\mathcal{D})$. This posterior distribution represents the updated knowledge about the model parameters given the observed data. Since the posterior distribution learned by Bayesian deep learning is a probability distribution, it can provide a measure of uncertainty about the model parameters. By taking into account the uncertainty in the model parameters, Bayesian deep learning can also estimate uncertainty for the model predictions.

A stochastic process is a collection of random variables $\{g(x)|x \in X\}$ for a given input dataset X. Based on the stochastic process, the definition of Gaussian Process is given as:

Definition 1. *A Gaussian Process (GP) is a random process that can be completely defined by its mean function $\mu(x)$ which represents the expected value of $g(x)$ and by its covariance function $\mathcal{K}(x_i, x_j) = \mathbb{E}[(g(x_i) - \mu(x_i))(g(x_j) - \mu(x_j))]$ which describes the covariance between the given input $g(x_i)$ and $g(x_j)$. Moreover, any finite set of the variables $\{g(x_i)\}_{i=1}^k$ will exhibit a joint multivariate Gaussian distribution.*

Given the training input $X = \{x_1, \cdots, x_n\}$ and $x_i \in \mathbb{R}^{d_0}$, the underlying assumption for the GP model applied for the observed data is that it is generated from a prior stochastic process, followed by the addition of independent Gaussian noise. Consider a simple linear model with ρ_0 basic functions $f(\cdot)$, the output $g(x)$ is computed as:

$$g(x) = b + \sum_{j=1}^{\rho_0} w_j f_j(x) \tag{10}$$

where w and b are the weight and bias of the basic functions, respectively. Assume that the Gaussian prior is put on both weight and bias with $w \sim \mathcal{N}(0, \sigma_w^2 \mathcal{I})$ and $b \sim \mathcal{N}(0, \sigma_b^2 \mathcal{I})$. Here, $\mathcal{I} \in \mathbb{R}^{n \times n}$ is the Identity matrix. Since the weight and bias parameters are considered to be independent and identically distributed (i.i.d.), the input variables x_i and x_j are independent when $i \neq j$, the function g(x) is a sum of i.i.d. terms. Hence, it can be deduced that any finite set of $g(x)$ derived from different input x is a joint multivariate Gaussian distribution, which aligns with the definition of a Gaussian Process. We can denote it as $g(x) \sim GP(\mu, \mathcal{K})$.

$$\begin{aligned} \mu(x) &= \mathbb{E}(g_i(x)) = 0 \\ \mathcal{K}(x_i, x_j) &= \sigma_b^2 + \sigma_w^2 C(x_i, x_j) \end{aligned} \tag{11}$$

where $C(x_i, x_j)$ is the covariance between the input x_i and x_j.

Neural Network Gaussian Process Imputator. The core of NNGPI for missing data imputation is learning a DL model by the Gaussian Process. In the case of a standard neural network, its performance heavily relies on hyperparameters such as the number of layers and the dimension of each layer. Additionally, as discussed previously, the inference of the model is based on the

learned deterministic weights, which may lead to the overfitting issue. In contrast, a general GP approach can mitigate these problems by sampling the weights of the model from a learned prior distribution. The hyperparameters used in the prior distribution are much fewer compared to those used in a standard neural network. Furthermore, the output is weighted by all potential posterior distributions of the parameters, rather than relying on a single deterministic weight, thereby alleviating the overfitting problem. However, the inherent limited approximation capacity of the basic function employed in the general GP restricts its ability to accurately model the complex patterns in the given dataset.

In order to better utilize the approximation ability of neural networks, neural network gaussian process (NNGP) is proposed in [23]. NNGP is a special category of Bayesian deep learning model that combines the strengths of both neural network and GP. From the neural network perspective, the hidden layer of NNGP is characterized by an infinite number of neurons. These infinite hidden neurons are constructed using a set of GP basis functions. By leveraging the universal representation learning capability of deep learning models, this deep-learning-based kernel offers enhanced flexibility in adapting to the underlying data. From the GP standpoint, NNGP is comprised of an infinite number of basis functions, enabling a significantly larger model space to capture the complex patterns within dataset. Notably, NNGP exhibits an analytical kernel function for specific neural network architectures, such as the ReLU activation function. This analytical kernel function implies that the training and inference processes of NNGP can be solved in closed form, similar to a regular GP.

We consider a L-layer neural network. The i-th component of the network output in layer l ($0 < l < L$) is computed as:

$$\begin{aligned} g_i^l(x) &= b_i^l + \sum_{j=1}^{\rho_l} w_{ij}^l f_j^l(x) \\ f_j^l(x) &= \phi(g_j^{l-1}(x)) \end{aligned} \tag{12}$$

Where ρ_l is the width of layers in layer l (i.e., the number of neurons in layer l) and $\phi(\cdot)$ is the non-linearity function. Assume that $g_j^{l-1}(x)$ represents a GP, which implies that $f_j^l(x)$ is also a GP. Considering that $g_i^l(x)$ is a summation of i.i.d. terms, we can apply the Central Limit Theorem [18]. Consequently, when the limit of ρ_l grows towards infinity, the distribution of $g_i^l(x)$ will approach a Gaussian distribution. Moreover, applying the multidimensional Central Limit Theorem, any finite set of $g_i^l(x)$ will exhibit a joint multivariate Gaussian distribution. This property precisely aligns with the definition of GP. As the input has a zero mean, the output is a GP as $g_i^l(x) \sim GP(0, \mathcal{K}^l)$ where $\mathcal{K}^l$ is defined as:

$$\begin{aligned} \mathcal{K}^l(x_p, x_q) &= \sigma_b^2 + \sigma_w^2 \mathbb{E}_{g_i^{l-1} \sim GP(0, \mathcal{K}^{l-1})} \left[\phi\left(g_i^{l-1}(x_p)\right) \phi\left(g_i^{l-1}(x_q)\right) \right] \\ &= \sigma_b^2 + \sigma_w^2 F_\phi(\mathcal{K}^{l-1}(x_p, x_q), \mathcal{K}^{l-1}(x_p, x_p), \mathcal{K}^{l-1}(x_q, x_q)) \end{aligned} \tag{13}$$

Here, $F_\phi(\cdot)$ is a function of the kernel matrices of the previous layer including $\mathcal{K}^{l-1}(x_p, x_q)$,$\mathcal{K}^{l-1}(x_p, x_p)$ and $\mathcal{K}^{l-1}(x_q, x_q)$. And the formulation of $F_\phi(\cdot)$ depends only on the non-linearity function $\phi(\cdot)$. For specific non-linearity functions, the recurrence relation can be analytically computed. For the widely used non-linearity function ReLU, its formulation is as follows:

$$\begin{aligned} &F_\phi(\mathcal{K}^{l-1}(x_p, x_q), \mathcal{K}^{l-1}(x_p, x_p), \mathcal{K}^{l-1}(x_q, x_q)) = \\ &\frac{\sqrt{\mathcal{K}^{l-1}(x_p, x_p)\mathcal{K}^{l-1}(x_q, x_q)}}{2\pi} \left(\sin\theta_{x_p,x_q}^{l-1} + \left(\pi - \theta_{x_p,x_q}^{l-1}\right) \cos\theta_{x_p,x_q}^{l-1} \right) \end{aligned} \tag{14}$$

where the definition of $\theta^l_{x_p,x_q}$ is as:

$$\theta^l_{x_p,x_q} = \cos^{-1}\left(\frac{\mathcal{K}^l(x_p,x_q)}{\sqrt{\mathcal{K}^l(x_p,x_p)\mathcal{K}^l(x_q,x_q)}}\right) \tag{15}$$

And assume $w^0_{ij} \sim \mathcal{N}(0,\sigma_w^2/d_0)$ and $b^0_j \sim \mathcal{N}(0,\sigma_b^2)$, then $\mathcal{K}^0$ is initialized as:

$$\mathcal{K}^0(x_p,x_q) = \sigma_b^2 + \sigma_w^2(x_p x_q) \tag{16}$$

Based on the above iterative process, a sequential set of computations can be executed to derive $\mathcal{K}^L$ for the GP that characterizes the final output of the model. Our goal is to make predictions for new inputs. The prediction for the distribution $g(x_*)$ corresponding to a test point x_* can be obtained by applying Equation 9, which can be done analytically. As the prior and the noise model are both in the Gaussian distribution, the output $P(g(x_*)|g(X))$ is also a Gaussian distribution with mean and variance:

$$\begin{aligned}\overline{\mu} &= \mathcal{K}^T_{X,x_*}(\mathcal{K}_{X,X} + \sigma_b^2\mathcal{I})^{-1}g(X)\\ \overline{\mathcal{K}} &= \mathcal{K}_{x_*,x_*} - \mathcal{K}^T_{X,x_*}(\mathcal{K}_{X,X} + \sigma_b^2\mathcal{I})^{-1}\mathcal{K}_{X,x_*}\end{aligned} \tag{17}$$

The predicted distribution for $P(g(x_*)|g(X))$ is obtained through simple matrix computations while still encompassing a fully Bayesian training approach for the deep neural network model. The selection of the GP prior, which is influenced by the depth of the neural network model, the type of the non-linearity function, variances of weights and biases, determines the form of the covariance function used.

3.3 Uncertainty-based Calibration

In this part, we first analyze the uncertainty estimation of NNGPI and then show the uncertainty-based data calibration technique. Based on NNGPI, the probability distribution over w and b induces a probability distribution over output prediction $g(x_*)$. Given a test data x_*, its predicted mean and variance can be calculated by Equation 17. A low variance indicates a close similarity between the test data and the training data. Therefore, the confidence interval of the prediction for x_* can be computed as follows:

$$[\overline{\mu}_{x_*} - \psi_\alpha\overline{\mathcal{K}}_{x_*},\ \ \overline{\mu}_{x_*} + \psi_\alpha\overline{\mathcal{K}}_{x_*}] \tag{18}$$

where $\overline{\mu}_{x_*}$ and $\overline{\mathcal{K}}_{x_*}$ is the predicted target and variance for x_*, respectively. ψ_α represents the Z-score associated with the confidence level α of $\mathcal{N}(0,1)$. The Z-score is the percentile value derived from the standard normal distribution and can be obtained from a statistical table. The confidence interval signifies the probability α that the true value falls within the specified range. Given a confidence level α, the above confidence interval is only positively related to the variance $\overline{\mathcal{K}}_{x_*}$. Hence, when the test data exhibits higher variance, it implies that the corresponding data points can potentially fall within a wider range, indicating a high uncertainty.

The GP uncertainty is strongly correlated with trained network prediction error as illustrated in previous work [8]. Therefore, we design an uncertainty-based data calibration approach to calibrate the dataset. Specifically, given a test data x_*, its target value in m-th iteration is calibrated by

$$t_*^{\{m\}} = (1 - \frac{\beta}{\overline{\mathcal{K}}_{x_*}})t_*^{\{m-1\}} + \frac{\beta}{\overline{\mathcal{K}}_{x_*}}\overline{\mu}_{x_*} \tag{19}$$

Where $\beta \in (0, 1)$ is a coefficient to balance the previous data and the new predicted data. In Equation 19, data is more likely to be updated by the new predicted data when the variance is low. Low variance indicates low uncertainty and high accuracy, making the predicted data more reliable for updating the existing dataset.

3.4 Learning Objectives

The learning objective of NNGPI which is designed for missing data imputation is formulated using the mean-squared-error (MSE) metric. It aims to measure the disparity between the prediction and the actual ground-truth label. It penalizes larger deviations more heavily, making it sensitive to outliers. Moreover, it offers a differentiable and smooth loss surface that facilitates neural network training. To be more specific, considering a training dataset with input samples denoted as $X = \{x_1, x_2, \cdots, x_n\}$ and their respective ground-truth labels $T = \{t_1, t_2, \cdots, t_n\}$, the loss function of NNGPI can be expressed as follows:

$$\mathcal{L}(X, T) = \frac{1}{n} \sum_{i=1}^{n} \left((g(x_i) - t_i)^2 \right) \tag{20}$$

3.5 Forward Propagation of NOMI

The overall forward propagation algorithm of NOMI is summarized in Algorithm 1. It takes as input the test sample x_*, training dataset, neighbor set size, threshold and batch size. To begin, the missing value in x_* is initialized, and a batch of training samples is selected (Lines 1 and 2). The NOMI algorithm executes multiple iterations until the predicted uncertainty falls below the given threshold τ. In each iteration, it performs retrieval module, NNGPI and uncertainty-based calibration sequentially (Lines 3 to 15). In the retrieval module, it augments both the training samples and test samples (Lines 4 to 6). It then calculates the pairwise correlation of data samples in each layer (Lines 7 to 9). NNGPI outputs both the predicted imputation value and the uncertainty (Lines 10 to 11). Next, the data is calibrated (Lines 12 and 13). If the learned uncertainty is below the given threshold, the algorithm terminates (Lines 14 to 15). Finally, the imputation returns (Line 16).

Due to the high computational cost associated with optimization on the entire dataset, the common practice in current learning methods (e.g., in [49, 54]) is to perform optimization on a batch of the dataset, and we also follow this line. The framework of multi-round imputation serves two purposes: 1) *Finding good neighbors*: The dataset after calibration becomes closer to the original distribution, resulting in improved neighbor information compared to the raw dataset. 2) *Reducing uncertainty*: The uncertainty gradually decreases as the number of iteration increases, and thus leads to an improved imputation.

Lemma 1. *Assume that the output uncertainty of the samples with missing values are in a Gaussian Distribution $\mathcal{N}(\mu, \sigma)$, then Algorithm 1 terminates at most $\log_{\varphi_\tau} \frac{1}{n \times \Delta}$ where Δ is the missing rate and $\varphi_\tau = P(x \leq \tau) = \int_{-\infty}^{\tau} \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}(\frac{x-\mu}{\sigma})^2} dx$.*

Proof Sketch. There are $n \times \Delta$ samples with missing values and the uncertainty is in Gaussian Distribution. Thus, there is a probability of $\varphi_\tau = P(x \leq \tau) = \int_{-\infty}^{\tau} \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}(\frac{x-\mu}{\sigma})^2} dx$ that one sample terminates. When the algorithm terminates, it follows the constraint that $n \times \Delta \times \varphi_\tau^\Omega < 1$. Thus, it runs for at most $\Omega < \log_{\varphi_\tau} \frac{1}{n \times \Delta}$ iterations.

4 THEORETICAL ANALYSIS

In this section, we show that NOMI can be formulated as an instance of the Expectation Maximization (EM) algorithm following [14] and then present the overall time complexity.

Algorithm 1: The Forward Propagation of NOMI.

Input: The test sample x_*, training dataset $\{X, T\}$, neighbor set size K, threshold τ, batch size n_B.

Output: The imputation $\tilde{x}_*$

Initialize the missing value in x_*

$\{X_B, T_B\} \leftarrow$ Randomly select from $\{X, T\}$ with size n_B

while *True* **do**

`// Retrieval phase.`

for $x_i \in X_B \cup x_*$ **do**

idx = top_rank $(S(x_i, X), K)$

$x_i = x_i || \{S^N(x_i, x_j) \times x_j, \forall j \in \text{idx}\}$

`// Imputation by NNGPI.`

for $l = 1, \cdots, L$ **do**

for $x_p, x_q \in X_B \cup x_*$ **do**

$\mathcal{K}^l(x_p, x_q) = \sigma_b^2 + \sigma_w^2 F_\phi(\mathcal{K}^{l-1}(x_p, x_q), \mathcal{K}^{l-1}(x_p, x_p), \mathcal{K}^{l-1}(x_q, x_q))$

$\overline{\mu}_{x_*} = \mathcal{K}^T_{X_B, x_*}(\mathcal{K}_{X_B, X_B} + \sigma_b^2 \mathcal{I})^{-1} g(X_B)$

$\overline{\mathcal{K}}_{x_*} = \mathcal{K}_{x_*, x_*} - \mathcal{K}^T_{X_B, x_*}(\mathcal{K}_{X_B, X_B} + \sigma_b^2 \mathcal{I})^{-1} \mathcal{K}_{X_B, x_*}$

`// Uncertainty-based calibration.`

$t_*^{\{m\}} = (1 - \frac{\beta}{\overline{\mathcal{K}}_{x_*}}) t_*^{\{m-1\}} + \frac{\beta}{\overline{\mathcal{K}}_{x_*}} \overline{\mu}_{x_*}$

$\tilde{x}_* = t_*^{\{m\}}$

`// Termination Check.`

if $\overline{\mathcal{K}}_{x_*} < \tau$ **then**

Break

return $\tilde{x}_*$

4.1 NOMI as an EM algorithm

In this part, we first introduce some basic concepts of the EM algorithm and then illustrate that our algorithm can be explained from the perspective of the EM algorithm. The EM algorithm is an iterative optimization method used for estimating parameters in probabilistic models with latent variables. Its core idea involves alternating between expectation (E) and maximization (M) steps to gradually optimize parameters. Given the observed dataset $\{x_i\}_{i=1}^n$, the latent variable $z \in \mathcal{Z}$ and model parameter θ, it aims to maximize the joint distribution over the observed dataset and latent variable given the model parameter, i.e., the likelihood function $P(x_i, z|\theta)$. Directly calculating the maximum value of the likelihood function is prohibitively expensive. Therefore, the EM algorithm aims to optimize the lower bound of the likelihood function. It uses the following logarithmic form and can be bounded by Jensen's inequality [32]:

$$\begin{aligned} \sum_{i=1}^{n} \log \sum_{z \in \mathcal{Z}} P(x_i, z|\theta) &= \sum_{i=1}^{n} \log \sum_{z \in \mathcal{Z}} Q(z) \frac{P(x_i, z|\theta)}{Q(z)} \\ \geq \sum_{i=1}^{n} \sum_{z \in \mathcal{Z}} Q(z) &\log \frac{P(x_i, z|\theta)}{Q(z)} \end{aligned} \tag{21}$$

where $Q(z)$ is the weight for the corresponding latent variable. As it aims to maximize the likelihood function, we need to optimize the lower bound of the function, i.e., $Q(z)\log\frac{P(x_i,z|\theta)}{Q(z)}$. According to Jensen's inequality, in Equation 21, equality holds if and only if $Q(z) = P(z|x_i, \theta)$.

In m-th iteration, given the observed data $\{x_i\}_{i=1}^n$, the posterior distribution of the latent variables $z \in \mathcal{Z}$ and the model parameter for the previous iteration $\theta^{\{m-1\}}$, the E-step calculates the expected value of the likelihood function as:

$$Q(\theta, \theta^{\{m-1\}}) = \sum_{i=1}^{n} \sum_{z\in\mathcal{Z}} \log P(z|x_i, \theta^{\{m-1\}}) \log P(x_i, z|\theta) \tag{22}$$

In the M-step, it maximizes the expected value of the likelihood function $Q(\theta, \theta^{\{m-1\}})$ to obtain the model parameter $\theta^{\{m\}}$.

$$\theta^{\{m\}} = \arg\max_{\theta} Q(\theta, \theta^{\{m-1\}}) \tag{23}$$

The gradient descent method is commonly employed to maximize the likelihood function by identifying points where the gradient equals zero, indicating the attainment of either the maximum or minimum value. In machine learning, dealing with large datasets can be computationally expensive when computing the full gradient. Hence, the stochastic EM algorithm is proposed as an alternative approach [39, 61]. The stochastic EM algorithm computes the gradient descent using a small batch of observations.

In the context of missing data imputation, the data matrix contains a certain proportion of missing values, while the true data matrix (known as the clean data matrix denoted as ζ) is not available. We treat the clean data matrix as the latent variable and data in the training dataset with its target $\{x_i, t_i\}_{i=1}^n$ as observed data. Therefore, EM aims to maximize the following likelihood:

$$\sum_{i=1}^{n} \int \log P(x_i, t_i, \zeta|\theta) d\zeta \tag{24}$$

The E-step corresponds to the uncertainty-based data calibration step within the framework of NOMI. In the E-step of the m-th iteration, it calculates the expected value of the likelihood function:

$$Q(\theta, \theta^{\{m-1\}}) = \sum_{i=1}^{n} \int P(\zeta|x_i, t_i, \theta^{\{m-1\}}) \log P(x_i, t_i, \zeta|\theta) d\zeta \tag{25}$$

where $P(\zeta|x_i, t_i, \theta^{\{m-1\}})$ is the posterior distribution of the clean data matrix ζ given the training dataset and the learned model parameter. In the stochastic EM algorithm, the computation of the posterior distribution $P(\zeta|x_i, t_i, \theta^{m-1})$ of the clean data matrix ζ is approximated using a δ-distribution. The δ-distribution concentrates its mass at the expectation of the posterior distribution $\zeta^{\{m-1\}}$ (obtained by the data calibration step) and has zero mass elsewhere. Thus, the E-step corresponds to the uncertainty-based data calibration step within the framework of NOMI and the E-step can be approximated as follows:

$$Q(\theta, \theta^{\{m-1\}}) = \sum_{i=1}^{n} \log P(x_i, t_i, \zeta^{\{m-1\}}|\theta) \tag{26}$$

The M-step corresponds to the NNGPI step in the framework of NOMI. In the M-step, the objective is to maximize the expected value of the likelihood function defined in Equation 26. In the stochastic EM algorithm, this function is optimized using stochastic gradient descent on the limited labeled data. In this context, $\theta^{\{m\}}$ represents all the parameters associated with the prior

distribution. And the model learns the optimal parameters $\theta^{\{m\}}$ by maximizing the expected value of the likelihood function.

By this reformulation, NOMI inherits the nice properties of the EM algorithm. The EM algorithm guarantees a progressive increase in the log-likelihood value during each iteration until convergence [14] and thus the imputation capacity of NNGPI increases with iteration. Multiple iterations allow both the NNGPI and the uncertainty-based calibration to incorporate the most recent reliable information to achieve high-quality imputation. Additionally, the uncertainty-based calibration facilitates the acquisition of a more reliable mini-batch gradient, aligning it better with the gradient derived from a fully observed clean dataset. This alignment enables an unbiased update of the stochastic EM algorithm, thus enhancing the overall effectiveness of the NNGPI and supporting the uncertainty-based calibration step.

4.2 Time Complexity Analysis

NOMI runs for multiple iterations. In each iteration, it runs the retrieval phase, the NNGPI and the uncertainty-based data calibration sequentially. In terms of the retrieval phase, we use HNSW [28] which has a time complexity of $O(nlog(n))$ for constructing the index. HNSW has a time complexity of $O(log(n))$ for neighbors search given one query sample. There are at most n data samples in the dataset that have missing values, therefore, the total query time complexity is $O(n \times log(n))$. Regarding the imputation phase, the forward propagation of the Neural Network Gaussian Process can be optimized to a time complexity of $O(n_B^2))$ where n_B is the batch size [59] as batch technique is used for model training. Note that, n_B is typically a relatively small number compared to n, and is set to the smaller one of 300 and the dataset size according to the experiments in Exp-12. For a test query, it needs to execute the model inference by matrix multiplication with a time complexity of $O(n_B^2))$. There are at most n test queries. Hence, the time complexity of NNGPI is $O(n \times n_B^2)$. The uncertainty-based data calibration phase takes $O(1)$ to calibrate one data sample and $O(n)$ for all data samples. Therefore, suppose that NOMI runs for Ω iterations, then the overall time complexity of NOMI is $O\left(\Omega \times \left(2 \times n \times log(n) + n \times n_B^2 + n\right)\right)$.

5 EXPERIMENTAL EVALUATION

In this section, we experimentally evaluate the performance of NOMI and a variety of baselines over 12 real-world datasets.

5.1 Dataset Description

We use 12 public datasets from the UCI repository [6] to conduct the experiments. The statistics information is summarized in Table 2. Following the recent experimental survey paper [34], our experiments mainly focus on the first 10 datasets. Additionally, we also evaluate the methods on two million-scale large datasets (i.e., *Retail* and *WISDM*). Following [21, 34, 57], after imputing missing data on the evaluated datasets, the downstream task is classification which aims to classify each sample based on the imputed features. The original datasets are fully observed. Hence, we use the following three different missing mechanisms to randomly remove values and construct the data matrix for missing data imputation.

Missing Mechanisms: There are three types of data missing mechanisms that are commonly used in the literature [34, 43], i.e., missing completely at random (MCAR), missing at random (MAR), and missing not at random (MNAR).

The missing data of MCAR is in the following form where the missing probability is only dependent on itself. Here, $X = X^o \cup X^m$ is the union of the observed data X^o and miss data X^m

$$P(M|X) = P(M|X^o, X^m) = P(M) \tag{27}$$

Table 2. Statistics of the datasets

Dataset	# of data sample	# numerical	# categorical
Wine	178	13	1
Heart	303	7	7
Breast	699	9	1
Car	1,728	0	7
Wireless	2,000	7	1
Abalone	4,177	8	0
Turkiye	5,820	0	33
Letter	20,000	0	16
Chess	28,056	0	7
Shuttle	43,500	0	10
Retail	1,067,371	5	1
WISDM	15,630,426	3	2

While missing data of MAR is conditioned on the observed data X^o, which is shown as follows:

$$P(M|X^o, X^m) = P(M|X^o) \tag{28}$$

By contrast, the missing data of MNAR is dependent on the missing data X^m. The missing pattern is in the following form:

$$P(M|X^o, X^m) = P(M|X^m) \tag{29}$$

Feature Encoding: The numerical features are encoded in the input using their original values. In handling categorical features, label encoding [17] assigns unique integers to categories, while one-hot encoding represents each category with a binary vector, ensuring binary distinctiveness [2]. Considering that 1) recent works [21, 33, 54, 57] commonly encode categorical features into unique integers in their official implementations, and 2) a significant portion of datasets represents the raw categorical features using unique integers (e.g., *Breast* and *Car*), we thus use the label encoding in our experiments for a fair comparison. Additionally, we also use the one-hot encoding in Exp-15 to compare the performance.

5.2 Experimental Setup

Baselines: we compare NOMI with the following 11 algorithms:

- MEAN [36] is a statistics-based imputation method that uses the mean value of the same feature for miss data imputation.
- KNNI [1] is a similarity-based imputation method uses its K nearest neighbors as input for missing data imputation.
- MissFI [46] is a regression-based imputation method that ensembles many tree models as random forest for missing data imputation.
- MICE [42] is a regression-based method that estimates missing data for multiple times and uses the average as the imputation.
- MFI [22] is a compression-based imputation method that decomposes the data matrix and reconstructs the matrix for imputation.
- VAEI [31] is a distribution-generation-based method that uses the variational AE as the backbone for missing data imputation.

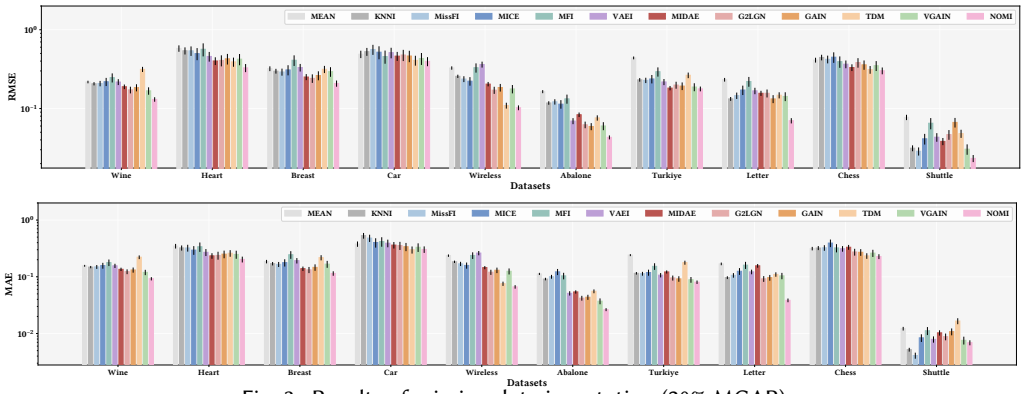


Fig. 3. Results of missing data imputation (20% MCAR)

- MIDAE [11] is a distribution-generation-based method that uses the deep denoising auto-encoder for missing data imputation.
- G2LGN [49] is a distribution-generation-based method which is based on GAN and captures the global and local information.
- GAIN [54] is a distribution-generation-based imputation method that uses the GAN as the backbone for imputation.
- TDM [57] is a distribution-matching-based imputation method that aims to minimize the optimal transport distance among imputed data batches in the latent space.
- VGAIN [34] is a distribution-generation-based method that is based on GAN and uses the VAE as the generator for imputation.

Metrics: Following the evaluation phase in [54], the result is first normalized, and then we use the root mean square error (RMSE) and the mean absolute error (MAE) that are defined as follows to measure the performance.

$$RMSE(X, \tilde{X}) = \sqrt{\frac{\sum_{j=1}^{d}\sum_{i=1}^{n}(1-m_{ij})(x_{ij}-\tilde{x}_{ij})^2}{\sum_{j=1}^{d}\sum_{i=1}^{n}(1-m_{ij})}} \tag{30}$$

$$MAE(X, \tilde{X}) = \frac{\sum_{j=1}^{d}\sum_{i=1}^{n}(1-m_{ij})\left|x_{ij}-\tilde{x}_{ij}\right|}{\sum_{j=1}^{d}\sum_{i=1}^{n}(1-m_{ij})} \tag{31}$$

For both metrics, smaller values indicate better performance.

Implementation Details: We follow the recent experimental survey paper [34]. For the KNNI method and our retrieval module, we set the neighbor size to 10 by default. We use the AKNN algorithm HNSW [28] for a fast neighbor retrieval. The batch size is set to the smaller of 300 or the dataset sizes. VGAIN and TDM run for 10000 epochs (or iterations) as in the original paper. For VGAIN, the parameters in the generator follow the settings in VAEI and other settings follow GAIN. MCAR with 20% missing data is the default missing mechanism. The NNGPI uses ReLU as the activation function. We totally train the NOMI for up to 3 iterations. The L_2 distance is used as the default metric for neighbor search. τ and β are set to 1.0 and 0.8 by default, respectively. The experiments are conducted on a machine with Intel(R) Xeon(R) Silver 4314 CPU, 504GB memory, and A5000 (GPU) under the framework of Pytorch.

5.3 Effectiveness Results

Exp-1: Performance under MCAR. Following the common setting of the recent works, we first test our algorithm under the MCAR mechanism with a missing rate of 20%. The RMSE and MAE

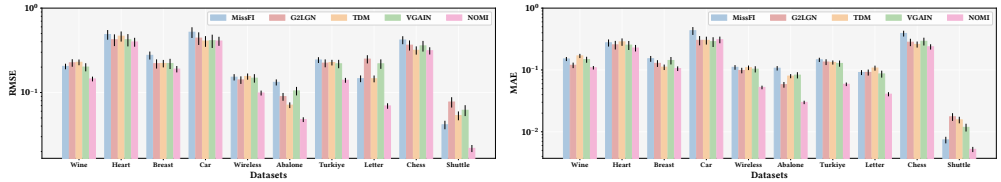


Fig. 4. Results of missing data imputation (20% MAR)

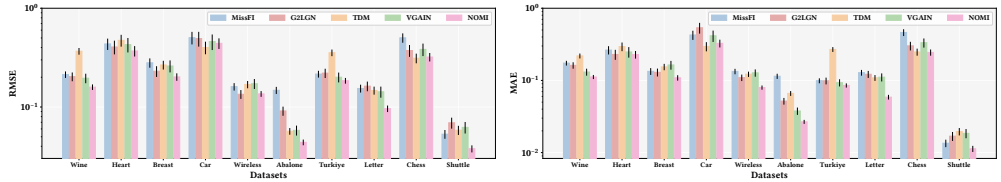


Fig. 5. Results of missing data imputation (20% MNAR)

results are illustrated in Figure 3. In general, distribution-modeling-based methods outperform both regression-based and compression-based approaches. Additionally, the aforementioned methods generally exhibit superior performance compared to both statistics-based and traditional similarity-based methods. Furthermore, the figure shows that NOMI exhibits superior performance compared to the baselines in terms of both RMSE and MAE. Specifically, NOMI effectively reduces the imputation error, as measured by RMSE, by 24.58% and 56.64% when compared to VGAIN and TDM, respectively. Additionally, NOMI reduces the imputation error, as measured by MAE, by 25.14% and 37.16% when compared to VGAIN and TDM, respectively. While TDM demonstrates favorable performance under *Car*, its effectiveness is hard to extend to other datasets. These results highlight the superior imputation performance of NOMI.

Exp-2: Performance under MAR and MNAR. In this part, we test the performance under both MAR and MNAR settings. Five algorithms are used for their high performance under MCAR including MissFI, G2LGN, TDM, VGAIN and NOMI. Missing data using the MAR mechanism is only dependent on the observed data. To model the correlation between missing data and observed data, a machine learning model is usually used. Following the works in [34, 37], the missing probability of the MAR mechanism is generated by the logistic model. First of all, a subset of fully observed features is randomly selected. After that, a logistic model is built based only on the fully observed features with random weights. The missing probability is calculated according to the trained logistic model and is re-scaled to obtain the desired proportion of missing data on those features. Missing data of MNAR is dependent on the missing data or the mask information in the whole dataset. Here, we also use a logistic model that depends on the masked values [37]. First of all, a subset of features that will have missing data is randomly selected. Then, the missing probability is generated by the logistic model based on the selected feature at random.

Consistent with Exp-1, the ratio of missing data of MAR and MNAR is also 20%. The results under MAR setting are illustrated in Figure 4, and the results under MNAR are shown in Figure 5. As depicted in the figures, NOMI shows a competitive performance under both the MAR and MNAR settings. In terms of the MAR, NOMI demonstrates a significant improvement in the imputation error, reducing the RMSE by 31.74% and 28.50% and reducing the MAE by 34.92% and 36.62% when compared to VGAIN and TDM, respectively. In the MNAR scenario, NOMI also exhibit a remarkable performance, resulting in a reduction of RMSE by 20.05% and 25.01%, as well as a reduction of MAE by 26.53% and 34.28%, when compared to VGAIN and TDM, respectively. These results exemplify the outstanding performance of NOMI under MAR and MNAR.

Exp-3: Downstream classification. In this part, we evaluate the downstream classification performance post-imputation. TDM and VGAIN are used for comparison as their high performance

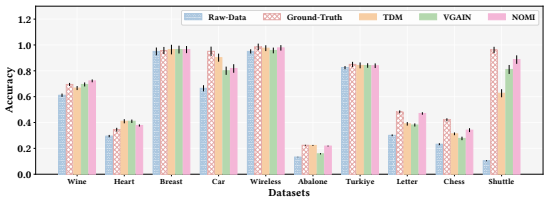


Fig. 6. Downstream classification performance

Table 3. Imputation time evaluation (in seconds)

	Wine	*Heart*	*Breast*	*Car*	*Wireless*	*Abalone*	*Turkiye*	*Letter*	*Chess*	*Shuttle*
TDM	1081	1144	1139	1145	1140	1141	1167	1160	1153	1176
VGAIN	234.9	222.8	224.4	77.26	223.7	218.5	212.9	195.2	205.3	210.0
NOMI	0.1+18.7	0.2+18.7	0.2+16.0	0.3+17.7	0.2+12.2	0.5+19.3	3.8+77.3	5.0+72.6	4.1+46.7	4.5+108.2

Table 4. Performance over large datasets

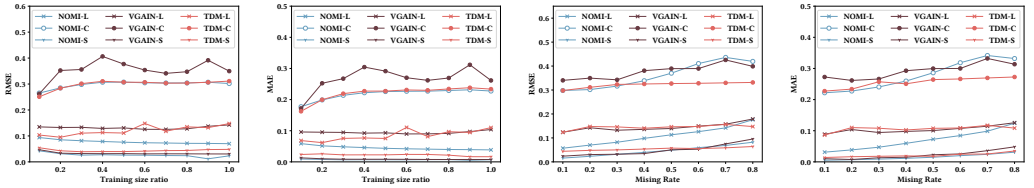
	Retail			*WISDM*		
	Time	RMSE	MAE	Time	RMSE	MAE
TDM	1190.8s	0.205 ± 0.019	0.126 ± 0.011	12072.6s	0.192 ± 0.018	0.116 ± 0.010
VGAIN	408.2s	0.220 ± 0.027	0.130 ± 0.017	7802.1s	0.214 ± 0.028	0.124 ± 0.015
NOMI	105.8s+205.2s	0.192 ± 0.019	0.121 ± 0.012	1166.3s+1456.2s	0.187 ± 0.020	0.117 ± 0.012
EDIT(NOMI)	214.3s	0.201 ± 0.022	0.129 ± 0.015	2293.5s	0.191 ± 0.021	0.119 ± 0.013

in Exp-1 and Exp-2. We also present the classification accuracy over the ground-truth data (original complete data) and raw data (incomplete data before imputation) for a comprehensive study. Here, missing data in raw data is substituted with the constant 0 following [21, 57]. We use the Support Vector Machine (SVM) [12] implemented from the scikit-learn package [40], and use 80% of the total data for training and the remaining 20% data for testing following [21]. The results are reported in Figure 6. The results indicate that datasets imputed by NOMI exhibit an average accuracy that is 15.48%, 3.03%, and 3.20% higher than raw data and datasets imputed by TDM and VGAIN, respectively. These findings highlight the superior performance of NOMI. Additionally, the results suggest that methods with lower RMSE (as displayed in Figure 3) generally exhibit better classification accuracy (as displayed in Figure 6).

5.4 Efficiency Results

Exp-4: Efficiency evaluation. In this section, we report the time required for training the imputation model. We conduct a comparative analysis among NOMI, VGAIN and TDM, considering their superior imputation accuracy observed in the previous evaluations and they are all learning-based imputation methods. The results are shown in Table 3. For NOMI, we report the time needed for neighbor retrieval and calibration plus the time for training NNGPI. The results show that NOMI exhibits a notably higher level of efficiency. NOMI can finish imputation within 120 seconds. When compared to TDM, NOMI improves the imputation efficiency by an average of 46.25×, with the highest improvement reaching up to 91.27× on the dataset *wireless*. Moreover, compared to VGAIN, NOMI enhances the imputation efficiency by an average factor of 8.22×, with the highest improvement reaching up to 17.91× on the dataset *wireless*. The results demonstrate the high efficiency of NOMI. The NNGPI possesses an analytical solution, and AKNN can efficiently locate local information, collectively contributing to the high efficiency of NOMI.

Exp-5: Performance over large datasets. In this part, we evaluate the performance over large datasets. Note that, to deal with large datasets, we split the test samples into sizes of 2048 and impute them together. The results are shown in Table 4. The results show that NOMI still has a strong performance compared to TDM and VGAIN over large datasets in terms of both accuracy and efficiency. Moreover, we integrate NOMI into EDIT [33] which is a framework designed specifically



(a) Varying training sizes (b) Varying training sizes (c) Varying missing rates (d) Varying missing rates

Fig. 7. Imputation results with different settings

for large dataset imputation. We use the samples selected by EDIT as the training dataset. The results suggest that EDIT can further boost the imputation efficiency with a slight accuracy degradation.

5.5 Imputation with Different Settings

Exp-6: Training dataset size. In this part, we investigate the impact of varying training sample sizes on the performance of imputation algorithms, including TDM, VGAIN and NOMI, which have demonstrated high performance in previous studies. Experiments are conducted on: *Letter*, *Chess*, and *Shuttle*, denoted as L, C, and S respectively, as they provide a large number of samples. We randomly select a portion of the complete dataset as the training dataset and apply the imputation algorithms. We consider different training sample ratios from 0 to 1, selecting a value every 0.1 for training. The resulting RMSE and MAE values are presented in Figure 7(a) and Figure 7(b) respectively. The results show that the performance of all three algorithms exhibits fluctuations across different training sizes. Notably, NOMI demonstrates competitive performance across various ratios of training sizes.

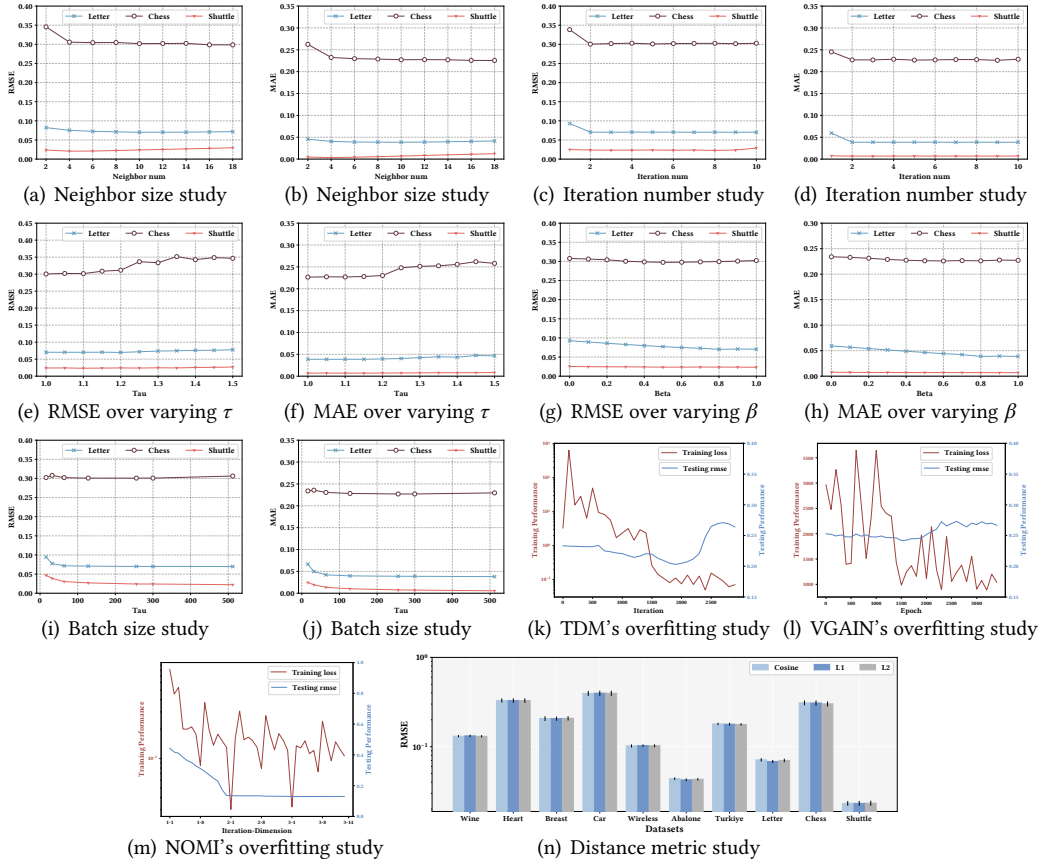
Exp-7: Missing rate. To study the effect of the missing rate on the imputation performance, we test our algorithm with a missing rate varying from 10% to 80%. Experiments are conducted on: *Letter*, *Chess*, and *Shuttle*, denoted as L, C, and S, respectively. RMSE and MAE results are presented in Figure 7(c) and Figure 7 (d). The figures indicate that NOMI consistently exhibits impressive performance, particularly when the missing rate is less than 35%. As the missing rate increases, the accuracy of imputation naturally declines. In scenarios with high missing rates, NOMI faces challenges in extracting accurate local information due to the substantial amount of missing data. However, despite this barrier, NOMI still shows a competitive imputation performance when compared to the baselines.

5.6 Hyperparameter and Robustness Evaluation

Exp-8: The number of neighbors selected (i.e., K). In this part, we explore the impact of the parameter K. We conduct experiments on the *Letter*, *Chess* and *Shuttle* datasets. We set neighbor sizes ranging from 2 to 18 and present the corresponding RMSE and MAE results in Figure 8(a) and Figure 8(b) at an interval of 2. The results indicate that as the number of neighbors increases, there is a noticeable decline in imputation error. At first, the decrease in error is significant and then it gradually stabilizes. NOMI with $K = 10$ can generally show a superior performance.

Exp-9: Iteration number. In this part, we examine the imputation accuracy across varying numbers of iterations. We conduct experiments with iteration numbers ranging from 1 to 10. The corresponding RMSE and MAE results are presented in Figure 8(c) and Figure 8(d), respectively. From the figure, it is evident that the imputation error decreases with an increasing number of iterations and eventually converges at approximately 3 iterations. The multiple iterations lead to an improvement in the quality of neighbors and contribute to the excellent imputation accuracy.

Exp-10: Sensitivity to the threshold τ. We evaluate the performance of NOMI across various thresholds. To ensure better control, we normalize the uncertainty within the range of 1 to 2. We vary the parameter τ from 1 to 1.5 and present the results in Figure 8(e) and Figure 8(f) respectively



(a) Neighbor size study (b) Neighbor size study (c) Iteration number study (d) Iteration number study

(e) RMSE over varying τ (f) MAE over varying τ (g) RMSE over varying β (h) MAE over varying β

(i) Batch size study (j) Batch size study (k) TDM's overfitting study (l) VGAIN's overfitting study

(m) NOMI's overfitting study (n) Distance metric study

Fig. 8. Robustness and hyperparameter evaluation

with an interval of 0.05. The figure indicates that the performance of NOMI decreases as τ increases. The performance with τ will inherently benefit from the multi-iterations design. Compared to other choices, NOMI with $\tau = 1$ generally demonstrates a good imputation accuracy.

Exp-11: Sensitivity to the coefficient β. We conduct a performance evaluation of NOMI across various values of the coefficient β. β plays a crucial role in balancing the predictions from the previous and current iterations. We examine the coefficient over a range from 0 to 1, reporting the results at intervals of 0.1 in Figure 8(g) and Figure 8(h). Notably, the figure illustrates that the RMSE and MAE initially decrease and then increase as β increases. Importantly, β of value 0.8 achieves the best performance among the values tested. The performance with β will intrinsically benefit from the calibration operation. Compared to other choices, NOMI with $\tau = 0.8$ generally demonstrates a superior imputation accuracy.

Exp-12: Training batch size. We evaluate the performance of NOMI using batch sizes set to 16, 32, 64, 128, 256, 300, and 512, respectively. The results are illustrated in Figure 8(i) and Figure 8(j). The figure shows that NOMI with a batch size of 300 generally demonstrates good performance among the sizes tested.

Exp-13: Overfitting. In this part, we illustrate the overfitting risk. Overfitting is characterized by an observation where the training error decreases while the test error first decreases then increases, indicating the challenges in generalization. We train the model using 50% of the total samples and the corresponding training loss and tested RMSE over the whole dataset are reported. We conduct

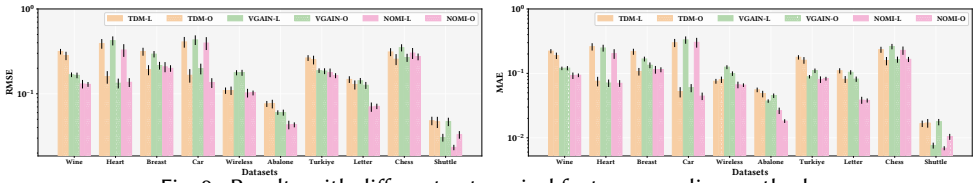


Fig. 9. Results with different categorical feature encoding methods

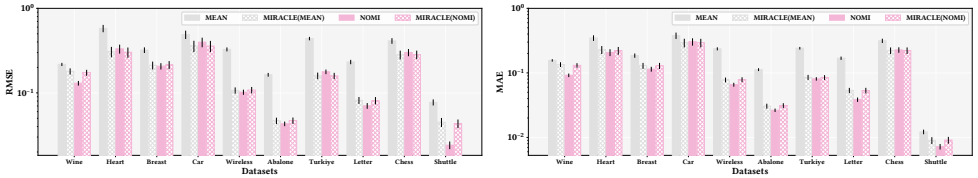


Fig. 10. Imputation result with the MIRACLE framework

experiments on the *Wine* dataset, and the results of TDM, VGAIN and NOMI are presented in Figure 8(k), (l) and (m) respectively. TDM, VGAIN and NOMI are trained over iteration, epoch, and iteration-dimension respectively as in their original works. The results show that the training loss of TDM and VGAIN progressively decreases while the test RMSE first decreases and then increases. In contrast, both training loss and test RMSE of NOMI progressively decrease.

Moreover, NOMI outperforms VGAIN and TDM in both MCAR and MNAR settings, with average RMSE improvements of 24.58%, 56.64%, 20.05% and 25.01%, respectively. VGAIN and TDM perform relatively worse under MCAR, suggesting challenges in generalizing to test data. This is because MCAR is an unstructured missing mechanism with missing patterns of test data differing from that of training data, thus indicating a potential overfitting issue.

These results collectively validate the robustness of NOMI to overfitting and align with the analysis provided in Section 1.

Exp-14: Distance metric. In this part, we evaluate the accuracy of imputation using three distinct distance metrics. We incorporate the cosine distance, L_1 distance and L_2 distance for selecting the local information. The results are depicted in Figure 8(n). The figure indicates that the performance is minimally affected by the different metrics. These results demonstrate the robustness of NOMI across various distance metrics.

5.7 Ablation Study

Exp-15: One-hot encoding. In this part, we evaluate the performance of models with different categorical feature encoding techniques, including label encoding and one-hot encoding. The results are depicted in Figure 9, where models suffixed with "-L" represent the label encoding, and those suffixed with "-O" signify one-hot encoding. The figure indicates that models utilizing one-hot encoded features generally exhibit superior performance compared to models using label encoding. This superiority may be attributed to two primary factors: 1) one-hot encoded data allows for rounding to the corresponding value after imputation, thereby mitigating bias; 2) label encoding encoded data implies certain magnitudes or distances between categories, which might mislead the model and thus harm the accuracy of the imputation.

Exp-16: MIRACLE. In this part, we study the framework of MIRACLE [21] which aims to impute data while preserving the causal structure. MIRACLE can be integrated with any seed model. We integrate MIRACLE with MEAN imputation and NOMI respectively, and present the results in Figure 10. The results reveal that NOMI maintains excellent performance, and integrating NOMI to MIRACLE does not surpass the standalone performance of NOMI. The reason may be two-fold: 1) the seed NOMI and downstream MIRACLE are independently optimized in the MIRACLE framework; 2) MIRACLE does not make good use of the local information.

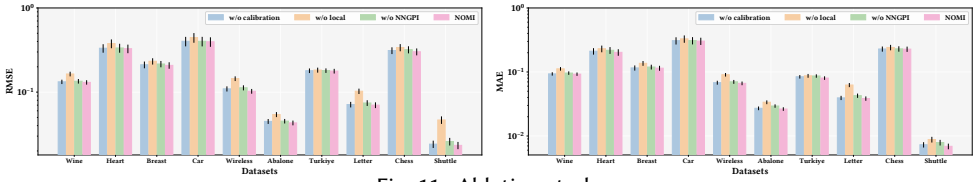


Fig. 11. Ablation study

Exp-17: Ablation study. In this part, we further investigate the impact of each individual component on the performance of NOMI. Our investigation focuses on three components proposed in this paper, i.e., the uncertainty-based calibration, the local information by the retrieval module and NNGPI. To analyze the effect of NNGPI, we substitute it with the generator used in VGAIN, and the results are presented in Figure 11. The figure reveals that all three components contribute to the high performance of NOMI. The retrieved local information plays a pivotal role in the overall framework and significantly enhances the imputation accuracy.

6 RELATED WORK

Statistics-based and similarity-based imputation algorithms. The statistics-based algorithms impute the missing data with some statistical values (e.g., the mean or mode values) for each feature that contains missing values [7, 36]. The similarity-based algorithms impute the missing data with close data. K nearest neighbor imputation (KNNI) [1] first selects K nearest neighbor of each sample that contains missing features. Hot deck imputation (HDI) [47] leverages the value from the most similar sample for imputation. Cold deck imputation (CDI) [16] uses an additional dataset for imputation.

Compression-based imputation algorithms. The soft-impute method [30] uses a soft threshold singular value decomposition (ST-SVD) to decompose the dataset with respect to a given threshold. Matrix factorization technique like the latent factor model [15, 22] is also employed to decompose the dataset matrix into two small matrices. PCAI is based on the principal component analysis [47].

Regression-based imputation algorithms. XGBoost imputation method (XGBI) [4] utilizes the XGBoost tree model for training. MissForest imputation (MissFI) [46] ensembles many tree models as random forests to train the parameters. MICE [42] utilizes the average value of the linear regression results to impute the missing values. Imputation via individual model (IIM) [55] learns a group of models and then uses the closest regression model for prediction. Multi-layer perceptron imputation (MLPI) [9] is based on the predictive power of the neural network for imputation.

Distribution-generation-based models for imputation. MIDAE is proposed in [11] that uses the deep denoising auto-encoder (DAE) to impute the missing data. Variational auto-encoder is used for the missing data imputation task in [31]. MIWAE is proposed in [29] that utilizs the importance-weighted autoencoder (IWAE) for the missing value imputation task. GINN [45] models the tabular data as a graph and combines the graph neural network into a GAN framework to impute the missing data. GAIN [54] utilizes the fully connected network as the backbone of the generator and discriminator and is trained in an adversarial manner to learn the latent distribution of missing data. VGAIN [34] combines the advantages of VAE and GAN. G2LGN [49] utilizes the GAN as the basic backbone and captures both the local and global features.

Distribution-matching-based models for imputation. These models utilize imputation techniques to ensure that the missing values are imputed in such a way that the empirical distributions of the two batches become aligned, thereby achieving a high degree of distributional closeness. The distance between the distributions of the two batch data is captured by the optimal transport. OTImputer [37] minimizes the optimal-transport-based cost function in the data space and TDM [57] minimizes the optimal-transport-based objective in the latent space.

7 CONCLUSION

In this paper, we explore the problem of missing data imputation. To enhance the imputation performance, we propose a novel model called NOMI that incorporates the retrieval module, NNGPI and an uncertainty-based calibration module. These modules are iteratively executed. The NNGPI combines both the predictive capacity of deep neural networks and the uncertainty estimation capacity of the Gaussian Process to predict both missing data and uncertainty while mitigating the overfitting issue. Additionally, we design an uncertainty-based calibration module that leverages the learned uncertainty to help the retrieval module obtain more reliable and accurate local information. Extensive experiments demonstrate the excellent performance of NOMI.

ACKNOWLEDGMENTS

Ying Zhang and Kai Wang are the joint corresponding authors. This work is partially supported by NSFC 62302294, NSFC U2241211, NSFC U20B2046, ARC DP210101393, ARC DP230101445, ARC LP210301046 and ARC FT210100303. We would like to thank the anonymous reviewers for their valuable and constructive feedback.

REFERENCES

[1] Naomi S Altman. 1992. An introduction to kernel and nearest-neighbor nonparametric regression. *The American Statistician* 46, 3 (1992), 175–185.

[2] Sikha Bagui, Debarghya Nandi, Subhash Bagui, and Robert Jamie White. 2021. Machine learning and deep learning for phishing email classification using one-hot encoding. *Journal of Computer Science* 17 (2021), 610–623.

[3] Steven L Brunton and J Nathan Kutz. 2022. *Data-driven science and engineering: Machine learning, dynamical systems, and control.* Cambridge University Press.

[4] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, August 13-17, 2016*, Balaji Krishnapuram, Mohak Shah, Alexander J. Smola, Charu C. Aggarwal, Dou Shen, and Rajeev Rastogi (Eds.). ACM, 785–794. https://doi.org/10.1145/2939672.2939785

[5] Kin-On Cheng, Ngai-Fong Law, and Wan-Chi Siu. 2012. Iterative bicluster-based least square framework for estimation of missing values in microarray gene expression data. *Pattern Recognit.* 45, 4 (2012), 1281–1289. https://doi.org/10.1016/J.PATCOG.2011.10.012

[6] Dheeru Dua and Casey Graff. 2017. UCI Machine Learning Repository. http://archive.ics.uci.edu/ml

[7] Alireza Farhangfar, Lukasz A Kurgan, and Witold Pedrycz. 2007. A novel framework for imputation of missing values in databases. *IEEE Transactions on Systems, Man, and Cybernetics-Part A: Systems and Humans* 37, 5 (2007), 692–709.

[8] Yarin Gal et al. 2016. Uncertainty in deep learning.

[9] Pedro J García-Laencina, José-Luis Sancho-Gómez, and Aníbal R Figueiras-Vidal. 2010. Pattern classification with missing data: a review. *Neural Computing and Applications* 19, 2 (2010), 263–282.

[10] Charles J Geyer. 1992. Practical markov chain monte carlo. *Statist. Sci.* (1992), 473–483.

[11] Lovedeep Gondara and Ke Wang. 2018. Mida: Multiple imputation using denoising autoencoders. In *Advances in Knowledge Discovery and Data Mining: 22nd Pacific-Asia Conference, PAKDD 2018, Melbourne, VIC, Australia, June 3-6, 2018, Proceedings, Part III 22*. Springer, 260–272.

[12] Marti A. Hearst, Susan T Dumais, Edgar Osuna, John Platt, and Bernhard Scholkopf. 1998. Support vector machines. *IEEE Intelligent Systems and their Applications* 13, 4 (1998), 18–28.

[13] Geoffrey E Hinton and Ruslan R Salakhutdinov. 2006. Reducing the dimensionality of data with neural networks. *Science* 313, 5786 (2006), 504–507.

[14] Suhang Wang Hongliang Chi, Cong Qi and Yao Ma. 2023. Active Learning for Graphs with Noisy Structures. In *Proceedings of the 19th International Workshop on Mining and Learning with Graphs (MLG)*.

[15] Rodolphe Jenatton, Nicolas Le Roux, Antoine Bordes, and Guillaume Obozinski. 2012. A latent factor model for highly multi-relational data. In *Advances in Neural Information Processing Systems 25: 26th Annual Conference on Neural Information Processing Systems 2012. Proceedings of a meeting held December 3-6, 2012, Lake Tahoe, Nevada, United States*, Peter L. Bartlett, Fernando C. N. Pereira, Christopher J. C. Burges, Léon Bottou, and Kilian Q. Weinberger (Eds.). 3176–3184. https://proceedings.neurips.cc/paper/2012/hash/0a1bf96b7165e962e90cb14648c9462d-Abstract.html

[16] José M Jerez, Ignacio Molina, Pedro J García-Laencina, Emilio Alba, Nuria Ribelles, Miguel Martín, and Leonardo Franco. 2010. Missing data imputation using statistical and machine learning methods in a real breast cancer problem.

Artificial Intelligence in Medicine 50, 2 (2010), 105–115.
[17] Bin-Bin Jia and Min-Ling Zhang. 2021. Multi-dimensional classification via sparse label encoding. In *International Conference on Machine Learning*. PMLR, 4917–4926.
[18] Oliver Johnson. 2004. *Information theory and the central limit theorem*. World Scientific.
[19] Sham M Kakade, Shai Shalev-Shwartz, and Ambuj Tewari. 2012. Regularization techniques for learning with matrices. *The Journal of Machine Learning Research* 13, 1 (2012), 1865–1890.
[20] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. 2012. ImageNet Classification with Deep Convolutional Neural Networks. In *Advances in Neural Information Processing Systems 25: 26th Annual Conference on Neural Information Processing Systems 2012. Proceedings of a meeting held December 3-6, 2012, Lake Tahoe, Nevada, United States*, Peter L. Bartlett, Fernando C. N. Pereira, Christopher J. C. Burges, Léon Bottou, and Kilian Q. Weinberger (Eds.). 1106–1114. https://proceedings.neurips.cc/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html
[21] Trent Kyono, Yao Zhang, Alexis Bellot, and Mihaela van der Schaar. 2021. Miracle: Causally-aware imputation via learning missing data mechanisms. *Advances in Neural Information Processing Systems* 34 (2021), 23806–23817.
[22] Daniel Lee and H Sebastian Seung. 2000. Algorithms for non-negative matrix factorization. *Advances in Neural Information Processing Systems* 13 (2000).
[23] Jaehoon Lee, Yasaman Bahri, Roman Novak, Samuel S Schoenholz, Jeffrey Pennington, and Jascha Sohl-Dickstein. 2018. Deep Neural Networks as Gaussian Processes. In *International Conference on Learning Representations*.
[24] Pei Li, Jaroslaw Szlichta, Michael Böhlen, and Divesh Srivastava. 2021. ABC of order dependencies. *The VLDB Journal* (2021), 1–25.
[25] Wei-Chao Lin and Chih-Fong Tsai. 2020. Missing value imputation: a review and analysis of the literature (2006–2017). *Artificial Intelligence Review* 53, 2 (2020), 1487–1509.
[26] Roderick JA Little and Donald B Rubin. 2019. *Statistical analysis with missing data*. Vol. 793. John Wiley & Sons.
[27] Siqiang Luo, Ben Kao, Xiaowei Wu, and Reynold Cheng. 2019. MPR - A Partitioning-Replication Framework for Multi-Processing kNN Search on Road Networks. In *35th IEEE International Conference on Data Engineering, ICDE 2019, Macao, China, April 8-11, 2019*. IEEE, 1310–1321. https://doi.org/10.1109/ICDE.2019.00119
[28] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2018), 824–836.
[29] Pierre-Alexandre Mattei and Jes Frellsen. 2019. MIWAE: Deep Generative Modelling and Imputation of Incomplete Data Sets. In *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA (Proceedings of Machine Learning Research, Vol. 97)*, Kamalika Chaudhuri and Ruslan Salakhutdinov (Eds.). PMLR, 4413–4423. http://proceedings.mlr.press/v97/mattei19a.html
[30] Rahul Mazumder, Trevor Hastie, and Robert Tibshirani. 2010. Spectral regularization algorithms for learning large incomplete matrices. *The Journal of Machine Learning Research* 11 (2010), 2287–2322.
[31] John T McCoy, Steve Kroon, and Lidia Auret. 2018. Variational autoencoders for missing data imputation with application to a simulated milling circuit. *IFAC-PapersOnLine* 51, 21 (2018), 141–146.
[32] Edward James McShane. 1937. Jensen's inequality. (1937).
[33] Xiaoye Miao, Yangyang Wu, Lu Chen, Yunjun Gao, Jun Wang, and Jianwei Yin. 2021. Efficient and effective data imputation with influence functions. *Proceedings of the VLDB Endowment* 15, 3 (2021), 624–632.
[34] Xiaoye Miao, Yangyang Wu, Lu Chen, Yunjun Gao, and Jianwei Yin. 2022. An Experimental Survey of Missing Data Imputation Algorithms. *IEEE Transactions on Knowledge and Data Engineering* (2022).
[35] Todd K Moon. 1996. The expectation-maximization algorithm. *IEEE Signal Processing Magazine* 13, 6 (1996), 47–60.
[36] Karel GM Moons, Rogier ART Donders, Theo Stijnen, and Frank E Harrell Jr. 2006. Using the outcome for imputation of missing predictor values was preferred. *Journal of Clinical Epidemiology* 59, 10 (2006), 1092–1101.
[37] Boris Muzellec, Julie Josse, Claire Boyer, and Marco Cuturi. 2020. Missing data imputation using optimal transport. In *International Conference on Machine Learning*. PMLR, 7130–7140.
[38] Vinod Nair and Geoffrey E. Hinton. 2010. Rectified Linear Units Improve Restricted Boltzmann Machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML-10), June 21-24, 2010, Haifa, Israel*, Johannes Fürnkranz and Thorsten Joachims (Eds.). Omnipress, 807–814. https://icml.cc/Conferences/2010/papers/432.pdf
[39] George Papandreou, Liang-Chieh Chen, Kevin P Murphy, and Alan L Yuille. 2015. Weakly-and semi-supervised learning of a deep convolutional network for semantic image segmentation. In *Proceedings of the IEEE International Conference on Computer Vision*. 1742–1750.
[40] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.
[41] Ricardo Cardoso Pereira, Miriam Seoane Santos, Pedro Pereira Rodrigues, and Pedro Henriques Abreu. 2020. Reviewing autoencoders for missing data imputation: Technical trends, applications and outcomes. *Journal of Artificial Intelligence Research* 69 (2020), 1255–1285.

[42] Patrick Royston and Ian R White. 2011. Multiple imputation by chained equations (MICE): implementation in Stata. *Journal of Statistical Software* 45 (2011), 1–20.
[43] Donald B Rubin. 1976. Inference and missing data. *Biometrika* 63, 3 (1976), 581–592.
[44] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. 1986. Learning representations by back-propagating errors. *Nature* 323, 6088 (1986), 533–536.
[45] Indro Spinelli, Simone Scardapane, and Aurelio Uncini. 2020. Missing data imputation with adversarially-trained graph convolutional networks. *Neural Networks* 129 (2020), 249–260.
[46] Daniel J Stekhoven and Peter Bühlmann. 2012. MissForest—non-parametric missing value imputation for mixed-type data. *Bioinformatics* 28, 1 (2012), 112–118.
[47] Bhekisipho Twala, Michelle Cartwright, and Martin Shepperd. 2005. Comparison of various methods for handling incomplete data in software engineering databases. In *2005 International Symposium on Empirical Software Engineering, 2005.* IEEE, 10–pp.
[48] Hao Wang and Dit-Yan Yeung. 2020. A survey on Bayesian deep learning. *ACM Computing Surveys (csur)* 53, 5 (2020), 1–37.
[49] Wei Wang, Yimeng Chai, and Yue Li. 2022. A Global to Local Guiding Network for Missing Data Imputation. In *ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP).* IEEE, 4058–4062.
[50] Andrew G Wilson and Pavel Izmailov. 2020. Bayesian deep learning and a probabilistic perspective of generalization. *Advances in Neural Information Processing Systems* 33 (2020), 4697–4708.
[51] Yangyang Wu, Xiaoye Miao, Yuchen Peng, Lu Chen, Yunjun Gao, and Jianwei Yin. 2022. An Interactive Data Imputation System. In *International Conference on Database Systems for Advanced Applications*. Springer, 495–499.
[52] Yangyang Wu, Jun Wang, Xiaoye Miao, Wenjia Wang, and Jianwei Yin. 2023. Differentiable and scalable generative adversarial models for data imputation. *IEEE Transactions on Knowledge and Data Engineering* (2023).
[53] Keyu Yang, Xin Ding, Yuanliang Zhang, Lu Chen, Baihua Zheng, and Yunjun Gao. 2019. Distributed similarity queries in metric spaces. *Data Science and Engineering* 4 (2019), 93–108.
[54] Jinsung Yoon, James Jordon, and Mihaela Schaar. 2018. Gain: Missing data imputation using generative adversarial nets. In *International Conference on Machine Learning*. PMLR, 5689–5698.
[55] Aoqian Zhang, Shaoxu Song, Yu Sun, and Jianmin Wang. 2019. Learning individual models for imputation. In *2019 IEEE 35th International Conference on Data Engineering (ICDE).* IEEE, 160–171.
[56] Shichao Zhang. 2012. Nearest neighbor selection for iteratively kNN imputation. *Journal of Systems and Software* 85, 11 (2012), 2541–2552.
[57] He Zhao, Ke Sun, Amir Dezfouli, and Edwin V Bonilla. 2023. Transformed Distribution Matching for Missing Value Imputation. In *International Conference on Machine Learning*. PMLR, 42159–42186.
[58] Junbo Zhao, Michael Mathieu, and Yann LeCun. 2016. Energy-based Generative Adversarial Networks. In *International Conference on Learning Representations*.
[59] Kangfei Zhao, Jeffrey Xu Yu, Zongyan He, Rui Li, and Hao Zhang. 2022. Lightweight and accurate cardinality estimation by neural network gaussian process. In *Proceedings of the 2022 International Conference on Management of Data*. 973–987.
[60] Zhengli Zhao, Sameer Singh, Honglak Lee, Zizhao Zhang, Augustus Odena, and Han Zhang. 2021. Improved Consistency Regularization for GANs. In *Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2021, Thirty-Third Conference on Innovative Applications of Artificial Intelligence, IAAI 2021, The Eleventh Symposium on Educational Advances in Artificial Intelligence, EAAI 2021, Virtual Event, February 2-9, 2021*. AAAI Press, 11033–11041. https://doi.org/10.1609/AAAI.V35I12.17317
[61] Rongda Zhu, Lingxiao Wang, Chengxiang Zhai, and Quanquan Gu. 2017. High-dimensional variance-reduced stochastic gradient expectation-maximization algorithm. In *International Conference on Machine Learning*. PMLR, 4180–4188.
[62] Zulun Zhu, Jiaying Peng, Jintang Li, Liang Chen, Qi Yu, and Siqiang Luo. 2022. Spiking Graph Convolutional Networks. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI 2022, Vienna, Austria, 23-29 July 2022*, Luc De Raedt (Ed.). ijcai.org, 2434–2440. https://doi.org/10.24963/IJCAI.2022/338

Received October 2023; accepted January 2024